
Augmentron: Scalable Multi-View Visual Augmentation for Robot Learning

Tanay Tandon^{1,2,*} Aseem Doriwala^{1,*}
Jade Choghari¹ Catherine Weaver¹ Pragna Mannam¹
¹Scale AI ²University of Pennsylvania
*Equal contribution

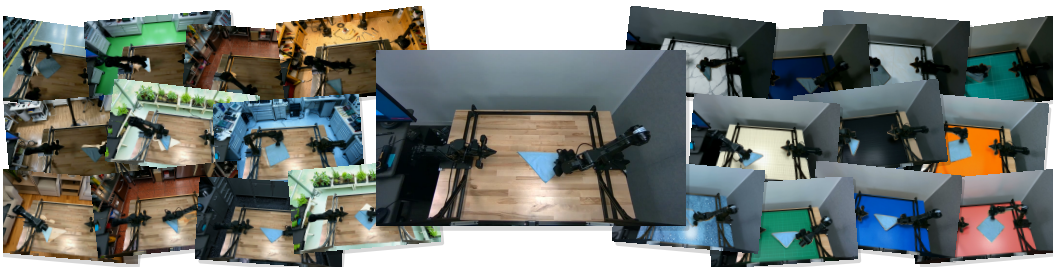


Figure 1: Environmental and workspace surface visual diversity produced by Augmentron: a geometry-aware pipeline that generates multi-view and temporally consistent data augmentation scalably for robotics manipulation datasets.

Abstract

Robot policies trained on visually narrow demonstrations can become sensitive to environmental features that are unrelated to the task. Existing generative augmentation methods can introduce visual diversity, but processing every frame and camera view independently is computationally expensive and may produce inconsistent observations across cameras and timestamps. We introduce Augmentron, a geometry-aware pipeline that generates reusable scene representations and renders them through recorded camera poses. This produces temporally and multi-view consistent environment and workspace variations while preserving the robot, task objects, actions, and annotations. Across three physical manipulation tasks, replacing half of the real world training dataset with Augmentron data improves model success across three tasks for both in out-of-distribution distractor objects, and out-of-distribution workspace variation evaluations. In a separate comparison against alternate augmentation techniques, Augmentron achieves the highest pooled success in all three conditions. The results show that introducing augmented data to a data training recipe can help policies generalize to dynamic real-world settings.

1 Introduction

Robot demonstration datasets are often visually narrow: the same environments, surfaces, and distractor configurations recur across many trajectories, even though many of these visual features provide little information relevant to the task. Policies trained on such data can fail when these

details change while the underlying task remains constant. In our evaluation, a real-only policy fine-tuned from $\pi_{0.5}$ [1] achieves 66.7% success on three in-distribution (ID) tasks, but success rate falls to 18.0% with out-of-distribution (OOD) distractor objects. This failure occurs despite broad vision-language-action pretraining [3, 9, 12].

Collecting new demonstrations across diverse settings is expensive and difficult to scale, making visual augmentation an attractive way to expand the coverage of existing data. Augmenting robot trajectories, however, is more difficult than editing individual images. The robot and task objects must be preserved, and visual changes must remain consistent over time and across camera views. Per-frame generative methods repeat segmentation and generation for every observation, while video-generation costs grow with trajectory length and camera count. Additionally, non-stationary robot wrist cameras introduce an additional challenge due to their continuously changing viewpoints.

To address these challenges, we introduce AUGMENTRON, a geometry-aware pipeline that generates a reusable representation of a visual domain that avoids editing frames independently. AUGMENTRON uses robot state and camera calibration to initialize robot masks and amortizes segmentation across the trajectory. Synthetically generated cubemaps and top-down scene images are then reused across fixed and wrist camera views to create consistent environment and working-surface variation. This design preserves the robot and task objects while avoiding repeated per-frame generative inference.

We evaluate AUGMENTRON across three physical manipulation tasks and three visual conditions. In a controlled replacement experiment, substituting half of the real world dataset with AUGMENTRON data improves model performance across all tasks in all three visual conditions. In a separate comparison of complete training volumes, AUGMENTRON achieves the highest overall task success rate in every condition, including consistently stronger results than RoboEngine [15]. AUGMENTRON also processes four-view demonstrations in $0.93\times$ their source-video duration, compared with $16.28\times$ for RoboEngine on the same source set and hardware.

2 Related Work

Visual augmentation for robot learning. Prior work introduces visual diversity through chroma keying, semantic image editing, and foreground-aware generation [13, 7, 15]. These methods show that varying task-irrelevant appearance can improve policy robustness, but typically process frames and camera views independently. AUGMENTRON instead reuses generated scene assets across an entire multi-view trajectory.

Video and multi-view generation. Video-diffusion methods condition generation on geometry, robot trajectories, reference images, or simulation to produce temporally coherent demonstrations [10, 8, 14, 6, 11]. Their computational cost grows with trajectory length and camera count, while consistency between generated views remains difficult. AUGMENTRON takes a rendering-oriented approach: calibration and robot state are used to project reusable generated appearances into synchronized fixed and wrist-camera views. We evaluate this tradeoff when fine-tuning a broadly pre-trained policy [1] under physical visual shifts.

3 Method

AUGMENTRON transforms the visual domain of a recorded demonstration while preserving its robot state, actions, language labels, and timing. As shown in Fig. 2, the pipeline has three stages: semantic mask generation, reusable scene representation generation, and geometry-aware rendering.

3.1 Semantic mask generation

We first separate protected regions (robot and task-relevant objects) from parts of the scene that may be modified during augmentation. The workspace surface is segmented separately so that it can be preserved during environment augmentation or replaced during workspace augmentation.

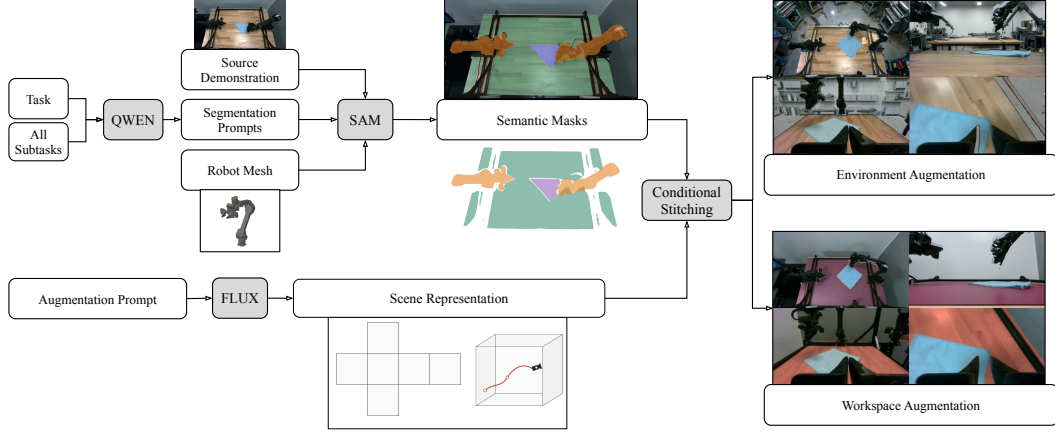


Figure 2: Overview of AUGMENTRON. Task annotations and robot geometry identify regions that must remain unchanged. A generated scene representation is then rendered and composited with the source demonstration to produce environment or workspace augmentations.

Robot masks. At each timestep, forward kinematics places the robot mesh in its recorded configuration. The mesh is projected into each camera using the calibrated camera parameters, producing a frame-specific robot mask. When necessary, this geometric estimate is refined using segmentation.

Task objects and workspace. An instruction-tuned language model translates task and subtask annotations into segmentation prompts, which are passed to SAM 3 [5]. SAM 3 identifies both task-relevant objects and the workspace surface. Fixed-camera surface masks can be reused over time because their image-space position remains stable. Robot wrist-camera masks are estimated separately due to their dynamic image-space position.

3.2 Scene representation generation

We construct an approximately foreground-free reference image by excluding protected pixels from sampled frames and computing a pixelwise temporal median. Because the robot and task objects move throughout the demonstration, locations occluded in one frame are often visible in another. The median therefore recovers much of the static scene while suppressing transient foreground content.

The reference image and an augmentation prompt are passed to the FLUX diffusion model family [2]. FLUX produces a reusable scene representation consisting of a static top-down scene image and a six-image cubemap. The static image captures the working area, while the cubemap represents the surrounding environment from each orthogonal direction. These assets are reused across the full trajectory rather than regenerating each observation independently.

3.3 Geometry-aware rendering and stitching

The generated scene representation is rendered using the recorded camera geometry and combined with protected pixels from the original demonstration. For a fixed camera, the static scene image is rendered once and reused across timesteps. The original robot and task-object pixels are then composited over the generated background using their semantic masks.

Wrist-camera views require rendering from a changing pose. Each camera pose is recovered from the recorded robot state and calibrated transforms. For workspace augmentation, the generated surface appearance is projected onto the calibrated table plane and rendered from each wrist-camera viewpoint. For environment augmentation, the cubemap is mapped onto the surrounding scene and rendered from the same poses. Protected region pixels are then composited over each rendering.

All synchronized camera streams use the same generated scene representation, preserving temporal and multi-view consistency while leaving the original actions and annotations unchanged.

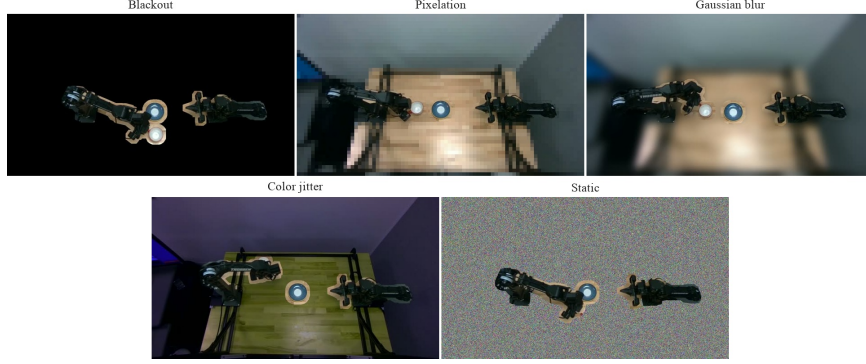


Figure 3: The five transformations used by the masked-noise baseline. Each corruption is applied outside the protected robot and task-object regions.

4 Experiments

Our experiments study whether training with AUGMENTRON data improves physical robustness over real-only training and how AUGMENTRON compares with generative and non-generative baselines.

4.1 Tasks and policies

We evaluate three manipulation tasks: placing a cup on a saucer, folding a towel, and inserting a flower into a vase. The training set contains 1,793 real demonstrations (9.16 hours) recorded with two fixed and two wrist cameras. The policy observes the high fixed view and both wrist views. All policies are fine-tuned from $\pi_{0.5}$ [1] using the LeRobot library [4]. The Real and AUGMENTRON 50/50 policies share the same 30k-step training configuration, while the full AUGMENTRON, RoboEngine, and Masked Noise policies share the same 45k-step configuration. Model initialization, batch size, camera inputs, and action representation are otherwise held fixed. The demonstrations contain two fixed and two wrist-camera streams. AUGMENTRON processes all four streams, while the policy observes only the high fixed view and both wrist views.

4.2 Training comparisons

Let R denote the original real robot demonstration corpus, E the AUGMENTRON environment-augmentation corpus, T the AUGMENTRON workspace-surface augmentation corpus, B the RoboEngine corpus, and N the masked-noise corpus. We define $1\times$ effective volume as one source-equivalent corpus, corresponding to the complete set of real robot demonstrations and approximately 9.16 hours of total recorded data.

We evaluate five recipes: **Real** ($1R$; $1\times$), **Augmentron 50/50** ($0.5R+0.25E+0.25T$; $1\times$), **Augmentron 33/67** ($1R+1E+1T$; $3\times$), **RoboEngine** ($1R+1B$; $2\times$) [15], and **Masked Noise** ($1R+1N$; $2\times$). In AUGMENTRON 50/50, the real and augmented halves use disjoint source episodes, so every source demonstration is represented exactly once. The remaining augmented recipes retain the full real corpus and add one or two source-equivalent corpora. Masked Noise assigns one of five protected-region-preserving transformations shown in Fig. 3 to each episode. Complete recipe and corruption details appear in Appendix Appendix A.

At 30k steps, we compare Real and AUGMENTRON 50/50 for a controlled replacement comparison because both effective data volume and training configuration are matched. We additionally report the 30k checkpoint of AUGMENTRON 33/67 as an exploratory scaling condition. At 45k steps, we compare AUGMENTRON 33/67, RoboEngine, and Masked Noise using the same optimization budget and training schedule. Effective data volume is not matched: AUGMENTRON uses $3\times$ volume, while RoboEngine and Masked Noise use $2\times$.

Table 1: Task and pooled success rates (%). Bold marks the highest observed task success rate within each condition and training-step group. Each task cell contains 50 rollouts and each pooled cell contains 150.

Steps	Recipe	Effective volume	ID				OOD distractors				OOD workspace			
			Cup $n = 50$	Towel $n = 50$	Vase $n = 50$	Pooled $n = 150$	Cup $n = 50$	Towel $n = 50$	Vase $n = 50$	Pooled $n = 150$	Cup $n = 50$	Towel $n = 50$	Vase $n = 50$	Pooled $n = 150$
30k	Real	1×	86	68	46	66.7	16	26	12	18.0	78	54	20	50.7
	Aug. 50/50	1×	72	88	70	76.7	52	52	32	45.3	80	94	28	67.3
	Aug. 33/67	3×	80	80	44	68.0	76	64	12	50.7	62	88	26	58.7
45k	Aug. 33/67	3×	82	72	60	71.3	74	80	44	66.0	70	84	28	60.7
	RoboEngine	2×	70	82	20	57.3	60	86	14	53.3	78	40	6	41.3
	Masked Noise	2×	86	94	26	68.7	62	88	6	52.0	62	76	6	48.0

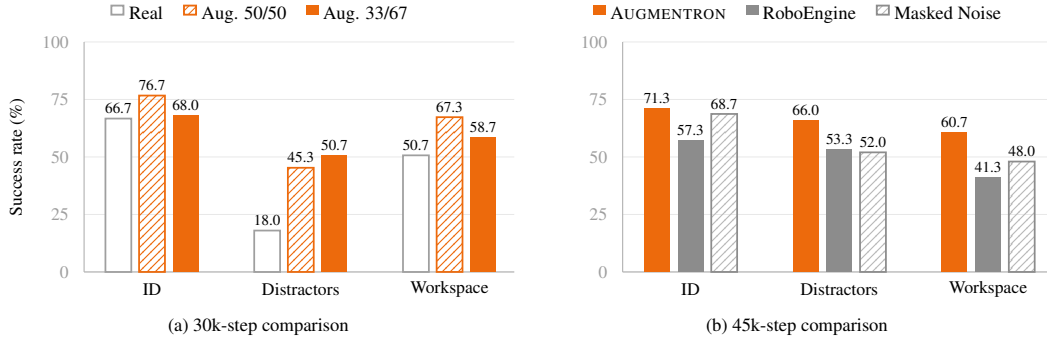


Figure 4: Pooled physical-rollout success. Left: 30k results; only Real versus AUGMENTRON 50/50 is matched in data volume and training schedule. Right: 45k recipe comparison; AUGMENTRON uses 3× effective volume and each baseline uses 2×. Each bar aggregates $n = 150$ rollouts.

4.3 Physical evaluation

Each policy is evaluated under three conditions: *ID*, using the original collection setup; *OOD distractors*, adding task-irrelevant objects; and *OOD workspace*, replacing the visible wooden surface with a steel reflective surface. We run 50 physical rollouts for each complete task–policy–condition experiment. We report task-level and pooled task success rates. Because there is one trained checkpoint per task and recipe, rollout counts measure evaluation uncertainty but not training-seed variation.

5 Results and Discussion

Table 1 summarizes the task-level and pooled results. We first analyze the controlled 30k-step replacement comparison, followed by the exploratory scaling comparison and the 45k-step baseline comparison.

5.1 Does Augmentron improve over real-only training?

The controlled comparison shows that replacing half of the real corpus with AUGMENTRON data improves performance across all three tested visual conditions (Fig. 4 and Table 1). Pooled success increases from 66.7% to 76.7% in ID, from 18.0% to 45.3% under OOD distractors, and from 50.7% to 67.3% under the workspace shift.

The largest improvement occurs under distractors. AUGMENTRON 50/50 improves cup-and-saucer success by 36 percentage points, towel folding by 26 points, and vase insertion by 20 points. This pattern is consistent with augmentation reducing sensitivity to visual features that repeatedly appear in the training data but are not required to complete the task.

5.2 Does augmented data ratio matter?

The 30k scaling comparison does not show a uniform benefit from the larger AUGMENTRON 33/67 recipe. Relative to AUGMENTRON 50/50, pooled success increases from 45.3% to 50.7% under OOD distractors, but decreases from 76.7% to 68.0% in ID and from 67.3% to 58.7% under the workspace shift.

The task-level results are also mixed. Under distractors, AUGMENTRON 33/67 improves cup-on-saucer and towel-folding performance, while vase insertion falls from 32% to 12%. This comparison is exploratory: AUGMENTRON 33/67 has a larger effective data volume, and its 30k checkpoint comes from a run scheduled for 45k steps. The observed differences therefore cannot be attributed solely to the augmented-data fraction. Additional matched mixtures and training seeds would be required to establish a scaling trend.

5.3 How does AUGMENTRON compare with the baselines?

At 45k steps, AUGMENTRON outperforms RoboEngine in the pooled comparison across all three conditions (Fig. 4). It achieves 71.3% in ID, 66.0% under OOD distractors, and 60.7% under the workspace variation, compared with 57.3%, 53.3%, and 41.3% for RoboEngine.

Masked Noise is a surprisingly competitive baseline, achieving 68.7%, 52.0%, and 48.0% across the same conditions. RoboEngine and Masked Noise produce similar pooled results under distribution shift, differing by 1.3 points under distractors and 6.7 points under the workspace shift. Both also show substantially smaller ID to OOD-distractor drops than the 30k Real policy: 4.0 points for RoboEngine and 16.7 points for Masked Noise, compared with 48.7 points for Real. Because this comparison crosses training configurations, it is descriptive rather than controlled, but it suggests that introducing visual variation can reduce sensitivity to distractors.

Among the complete 45k recipes, AUGMENTRON achieves the strongest pooled results in every condition. These findings are consistent with a compound benefit from its segmentation, richer scene representations, and temporal and multi-view consistency. However, these components are evaluated jointly, and AUGMENTRON uses $3\times$ effective data volume compared with $2\times$ for the baselines. The results therefore support the complete AUGMENTRON recipe without isolating the effect of any individual component.

5.4 Runtime

We measure end-to-end augmentation time on the same 30-minute, 104-episode source set using one eight-H100 node. AUGMENTRON processes four camera views in 0.47 hours, corresponding to $0.93\times$ the duration of the source videos. RoboEngine requires 8.14 hours, or $16.28\times$ source duration. A measured one-view Cosmos-Transfer1 run [11] requires 10.31 hours ($20.62\times$). Extrapolating its per-view stages to four views gives 23.79 hours ($47.57\times$).

AUGMENTRON avoids repeated generative inference for every frame. Once an appearance asset has been generated, longer trajectories primarily add rendering and compositing costs. This cost structure is particularly useful for long demonstrations containing several synchronized camera streams.

6 Conclusion

We presented AUGMENTRON, a geometry-aware pipeline for producing temporally and multi-view consistent visual augmentations of robot demonstrations. Instead of invoking a generative model independently for every frame, AUGMENTRON reuses edited scene representations and renders them through the recorded camera poses. This allows four-camera demonstrations to be processed in near real time ($0.93\times$ the source video duration).

Replacing part of the real training corpus with AUGMENTRON data improved pooled performance across all three evaluation conditions relative to real-only training. The pipeline also generated multi-view augmentations substantially faster than the per-frame generative baselines evaluated in this work. Although comparisons with other augmentation methods remain task-dependent, these results show that reusable, geometry-grounded scene assets can provide useful visual diversity at low processing cost.

Future work should evaluate additional tasks and visual shifts, train policies across multiple seeds, and separately control for dataset size and optimization exposure. Our results support AUGMENTRON as an efficient and effective augmentation approach in the settings studied, but do not establish universal superiority over alternative methods.

References

- [1] Kevin Black et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Conference on Robot Learning*, 2025.
- [2] Black Forest Labs. Flux.1 text-to-image models. *arXiv preprint arXiv:2408.00000*, 2024.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Chormanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [4] Remi Cadene, Simon Alibert, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouédec, and Thomas Wolf. Lerobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*, 2026.
- [5] Nicolas Carion et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.
- [6] Justin Chen, I-Chun Arthur Liu, Gaurav S. Sukhatme, and Daniel Seita. Craft: Video diffusion for bimanual robot data generation. *arXiv preprint arXiv:2604.03552*, 2026.
- [7] Zoey Chen, Zhao Mandi, Homanga Bharadhwaj, Mohit Sharma, Shuran Song, Abhinav Gupta, and Vikash Kumar. Semantically controllable augmentations for generalizable robot learning. *arXiv preprint arXiv:2409.00951*, 2024.
- [8] Zibin Dong et al. Emma: Generalizing real-world robot manipulation via generative visual transfer. *arXiv preprint arXiv:2509.22407*, 2025.
- [9] Moo Jin Kim, Karl Karlinsky, Homer Walke, Kevin Chen, Archit Sharma, Sergey Levine, and Chelsea Finn. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [10] Lijun Liu et al. Robottransfer: Controllable geometry-consistent video diffusion for manipulation policy transfer. *arXiv preprint arXiv:2505.23171*, 2025.
- [11] NVIDIA et al. Cosmos-transfer1: Conditional world generation with adaptive multimodal control. *arXiv preprint arXiv:2503.14492*, 2025.
- [12] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Karlinsky, Oier Mees, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [13] Eugene Teoh, Sumit Patidar, Xingyu Ma, and Stephen James. Green screen augmentation enables scene generalisation in robotic manipulation. *arXiv preprint arXiv:2407.07868*, 2024.
- [14] Bohan Wang et al. Robovip: Multi-view video generation with visual identity prompting augments robot manipulation. *arXiv preprint arXiv:2601.05241*, 2026.
- [15] Chengbo Yuan, Suraj Joshi, Shaoting Zhu, Hang Su, Hang Zhao, and Yang Gao. Roboengine: Plug-and-play robot data augmentation with semantic robot segmentation and background generation. *arXiv preprint arXiv:2503.18738*, 2025.

A Implementation and Evaluation Details

A.1 Mask generation

Robot masks. Robot masks are generated by placing the calibrated robot mesh at each recorded joint configuration and projecting it into each camera view. The experiments use these projected masks directly, without learned refinement or additional dilation.

Task-object masks. Qwen/Qwen3-4B-Instruct-2507 converts task and subgoal annotations into multiple SAM 3 prompts [5]. For fixed cameras, four sampled frames are tiled into a temporal mosaic and segmented together. Wrist-camera object masks are estimated separately for each view and frame because the camera and task objects can move substantially in image space. Task-object masks are dilated by six pixels before compositing to reduce the risk of modifying object boundaries.

Workspace masks. Fixed-camera workspace masks are estimated at anchor frames and aggregated over temporal windows. Wrist-camera workspace masks are generated from four-frame temporal mosaics because the surface position changes in image space as the arm moves.

A.2 Appearance generation and rendering

For each fixed camera, Augmentron constructs a reference image from up to 96 evenly spaced trajectory frames. Pixels covered by robot or task-object masks are excluded, and a per-pixel temporal median is computed from the remaining observations. Because the robot and objects move during a demonstration, regions occluded in one frame are often visible in another. Regions that remain unobserved are completed by the image-editing model.

We use black-forest-labs/FLUX.2-klein-4B [2] with four inference steps, guidance scale 1.0, and a maximum image dimension of 1024 pixels. The resulting scene representation consists of an edited static scene image and a six-face room representation containing four walls, a floor, and a ceiling.

For environment augmentation, the room representation is rendered through each recorded wrist-camera pose. For workspace augmentation, a sampled texture is anchored to the calibrated workspace plane and projected into each view. The original robot and task-object pixels are then composited over the rendered scene. The same generated assets are reused throughout a trajectory, preserving temporal and multi-view consistency.

Augmentron processes two fixed and two wrist-camera views. The policy consumes the high fixed view and both wrist views; the low fixed view is augmented but is not used as a policy input.

A.3 Training recipe construction

Let R denote the real corpus, E the Augmentron environment-augmentation corpus, T the Augmentron workspace-augmentation corpus, B the RoboEngine corpus, and N the masked-noise corpus. One source-equivalent corpus is denoted by $1\times$ effective volume.

- **Real** ($1R$; $1\times$): the complete real corpus.
- **Augmentron 50/50** ($0.5R + 0.25E + 0.25T$; $1\times$): the source episodes are divided into two disjoint halves. One half retains its real observations, while the other supplies the trajectories used for environment and workspace augmentation. Every source episode is therefore represented exactly once.
- **Augmentron 33/67** ($1R + 1E + 1T$; $3\times$): the complete real corpus plus one source-equivalent environment corpus and one source-equivalent workspace corpus.
- **RoboEngine** ($1R + 1B$; $2\times$): the complete real corpus plus one source-equivalent corpus generated with RoboEngine [15].
- **Masked Noise** ($1R + 1N$; $2\times$): the complete real corpus plus one corrupted copy of every source trajectory.

All recipes retain coverage of every original source demonstration. They differ in visual composition and effective data volume.

A.4 Dataset and policy training

The real corpus contains 1,793 demonstrations totaling approximately 9.16 hours: 527 cup-and-saucer demonstrations, 868 towel-folding demonstrations, and 398 vase-insertion demonstrations. The complete corpus is stored in the LeRobotDataset format [4]. Images are recorded at 1280×720 and resized to 224×224 for training.

All policies are fine-tuned from the same $\pi_{0.5}$ initialization [1]. The policy receives three camera views and a 14-dimensional robot state and predicts 14-dimensional actions. Actions are represented relative to the current state, except for the gripper dimension.

We train the complete model, including the vision encoder, using AdamW with a learning rate of 2.5×10^{-5} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.01, and gradient clipping at 1.0. The learning rate follows a cosine schedule with 1,000 warmup steps and a minimum of 2.5×10^{-6} . Training uses a batch size of 32 on each of eight ranks, giving a global batch size of 256. All runs use seed 1000.

The controlled Real and Augmentron 50/50 runs use a 30k-step cosine-decay horizon. The reported 30k Augmentron 33/67 result is an intermediate checkpoint from a run scheduled for 45k steps and therefore follows a 45k-step decay schedule. All policies in the 45k baseline comparison use the 45k-step schedule. We train one checkpoint for each task and recipe.

A.5 Masked-noise baseline

Masked Noise applies one of five transformations outside the protected robot and task-object regions:

- random RGB static;
- pixelation using 16-pixel blocks;
- Gaussian blur with $\sigma = 18$;
- complete blackout; and
- color jitter.

A corruption family is assigned deterministically to each episode and retained across all frames and policy views. Assignments are balanced by frame count within each task. Random parameters are generated deterministically from the episode, camera view, and frame identifiers.

A.6 Physical evaluation protocol

Each policy is evaluated under three conditions: *ID*, using the original collection setup; *OOD distractors*, with task-irrelevant objects added to the scene; and *OOD workspace*, with the visible wooden working surface replaced by steel.

We conduct 50 physical rollouts for each complete task–policy–condition cell, yielding 150 rollouts for each pooled result. Starting configurations are manually reset but are not paired across policies. Each rollout lasts at most 1,200 control steps, corresponding to 40 seconds at 30 Hz. The operator assigns a score from 0 to 3 to denote progress, and a score of 3 means task success. Trials interrupted by robot or operator faults are voided and repeated.

A.7 Runtime benchmark protocol

Runtime is measured on the same 30-minute cup-and-saucer source subset, containing 104 episodes, 54,000 frames at 30 Hz, and four camera views. Each method is run on one a3.megagpu.8g node with eight NVIDIA H100 GPUs. Timing begins with cold model construction and ends after the local output audit. It excludes queueing, container-image retrieval, credential and GPU gates, artifact staging, inter-boot delays, and result publication.

Augmentron processes the source data in 0.47 hours, or $0.93 \times$ its source-video duration. Robo-Engine requires 8.14 hours, or $16.28 \times$ source duration. A measured single-view Cosmos-Transfer1 run requires 10.31 hours ($20.62 \times$). Extrapolating its per-view stages gives an estimated four-view runtime of 23.79 hours ($47.57 \times$); this four-view value was not measured directly.